Computer Assignment 2: Filtering Lab

1. Complete the FIR filtering function and turn in a printout of your code. Do not use built-in functions or vector operations. Use `for` loops where needed. Write the filtering function using a single for loop.

Refer to end of document for function code.

2. Let the impulse response be h=[1,2,3,4,4,3,2,1] and prove that the code is correct by inputting an impulse sequence [1,0,0,0,0,⋯,0] and verifying that the output is the impulse response h[n]. You will need to write a script that calls your filtering code for each sample in the impulse sequence.

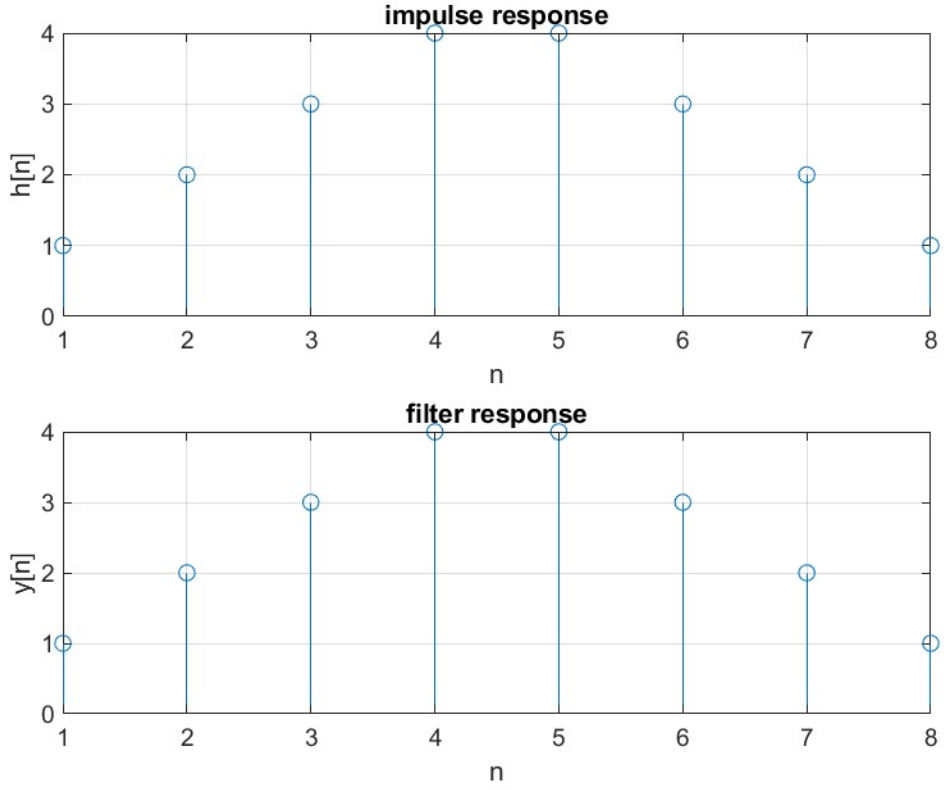


Figure 1: Plots showing the impulse response and the filter response.

3. Let the impulse response be h=[0.0625,0.2764,0.4182,0.2764,0.0625] and the input be x[n]=cos(2*pi*0.422n).
    1. Compute the filter output y[n] for n=0,1,2,⋯,20 and show that after an initial transient the steady state value of the output is zero.

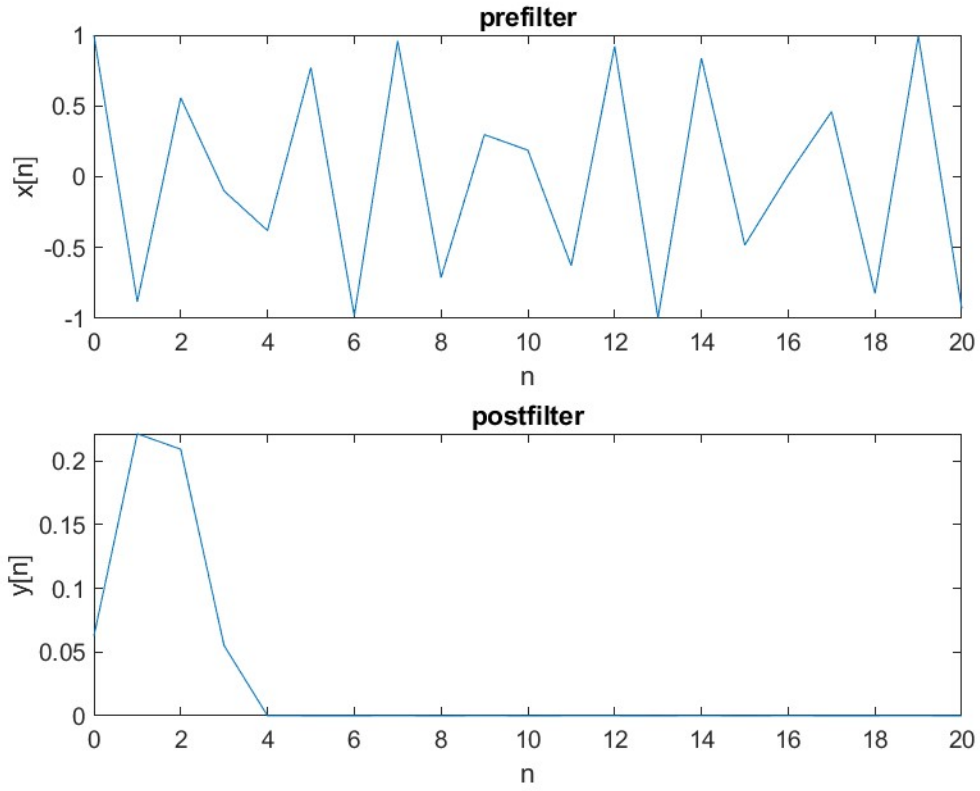


Figure 2: Plots showing that after the initial transient the output becomes zero.

    2. Using the `freqz` function, plot the magnitude response assuming the sample rate is Fs=1 sample/second.

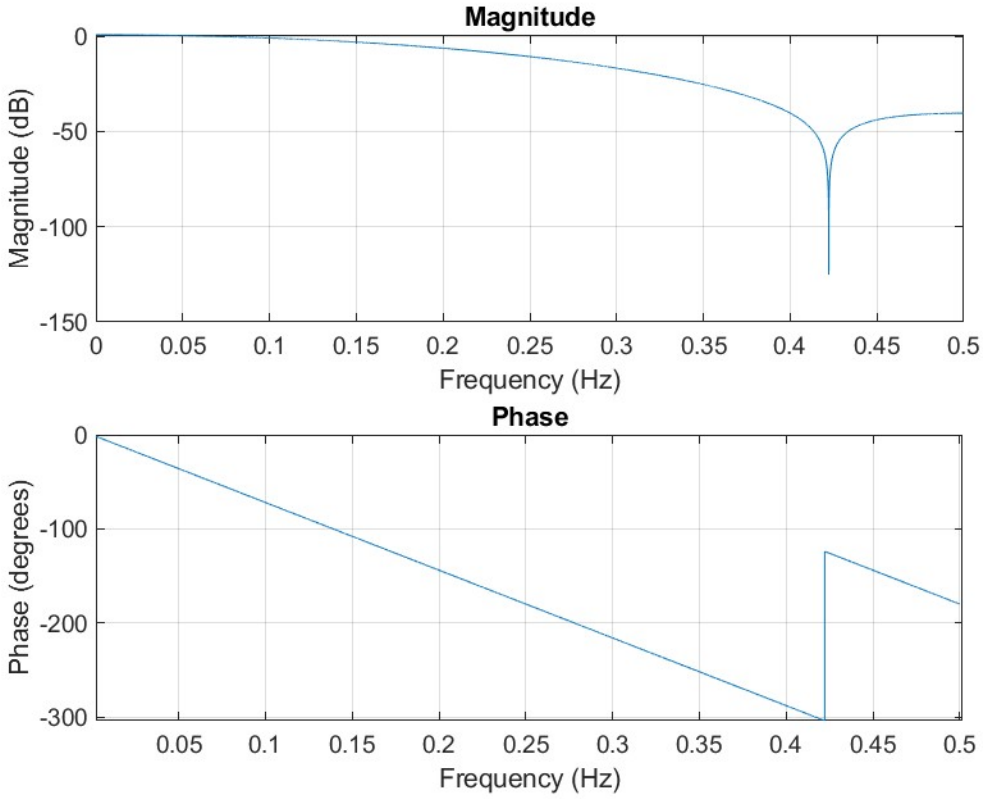


Figure 3: Magnitude and phase response of the filter

3. Using the magnitude response, explain why the steady state output is zero.

Because at the frequency the input function is set to the magnitude is about -120dB. Thus, the input cosine wave is completely attenuated.

4. Design a band-stop filter by combining all-pass and low-pass filters.
    1. Show how to compute the impulse response.

pick start and stop frequencies

LPF1

$f_{pass} = 1000Hz$

$f_{stop} = 1100Hz$

LPF2

$f_{pass} = 2000Hz$

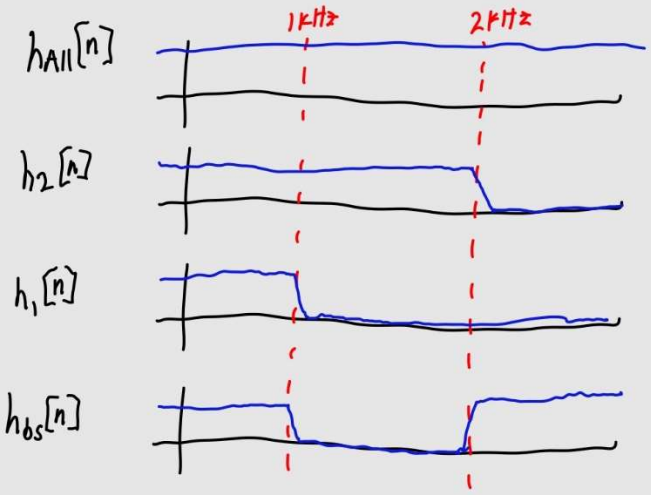
$f_{stop} = 2100Hz$

then using the formula to generate an impulse response with frequencies

$$h[n] = \frac{(2f_1)\sin(2\pi f_1 n)\sin(2\pi f_2 n)}{(2\pi f_1 n)(2\pi f_2 n)}$$

Using $h_1[n]$, $h_2[n]$, and an all pass filter you can create a bandstop filter

$$h_{bs}[n] = h_{All}[n] - h_2[n] + h_1[n]$$



2. Make a plot of the filter magnitude response.

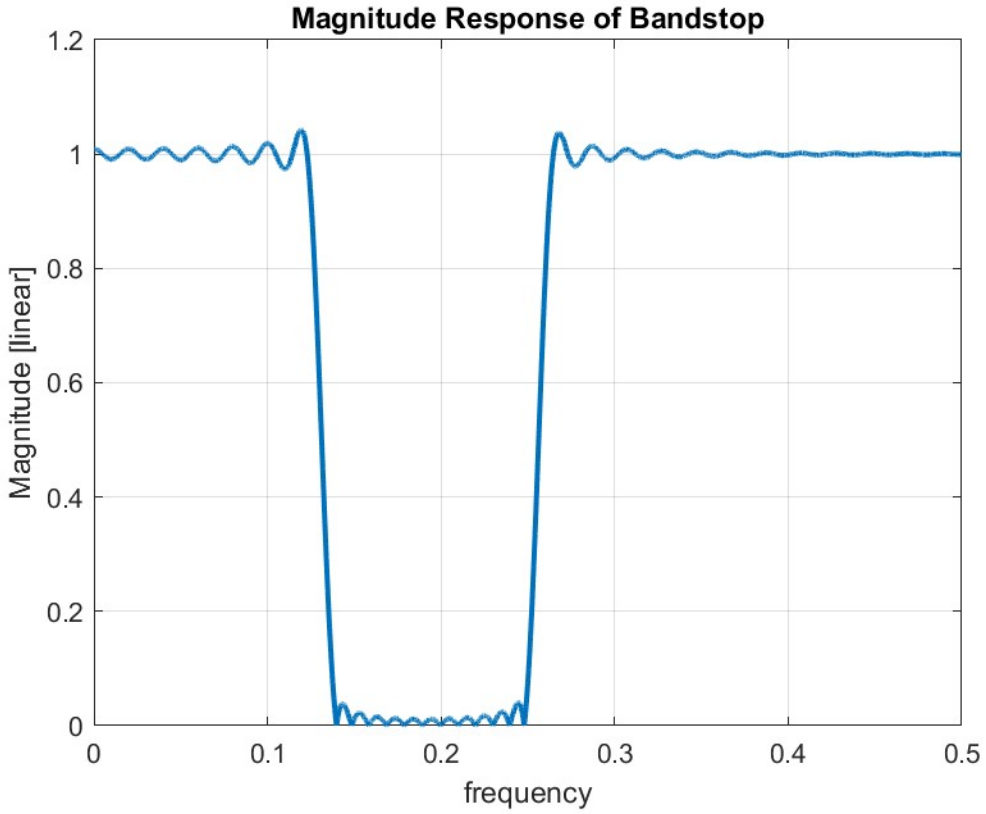


Figure 5: plot of the bandstop(linear) frequency response.

5. Complete the IIR filtering function and turn in a printout of your code. Do not use built-in functions or vector operations. Use `for` loops where needed. Write the filtering function using a single for loop for the feed forward section and a single for loop for the feedback section.

Refer to end of document for function code

6. Use the filter coefficients for the shelving filter and process the signal on the shelving filter web page. Use the original audio as the input signal rather than the whole song.

1. Make spectrogram plots of the input and output to illustrate that the shelving filter performs as expected.

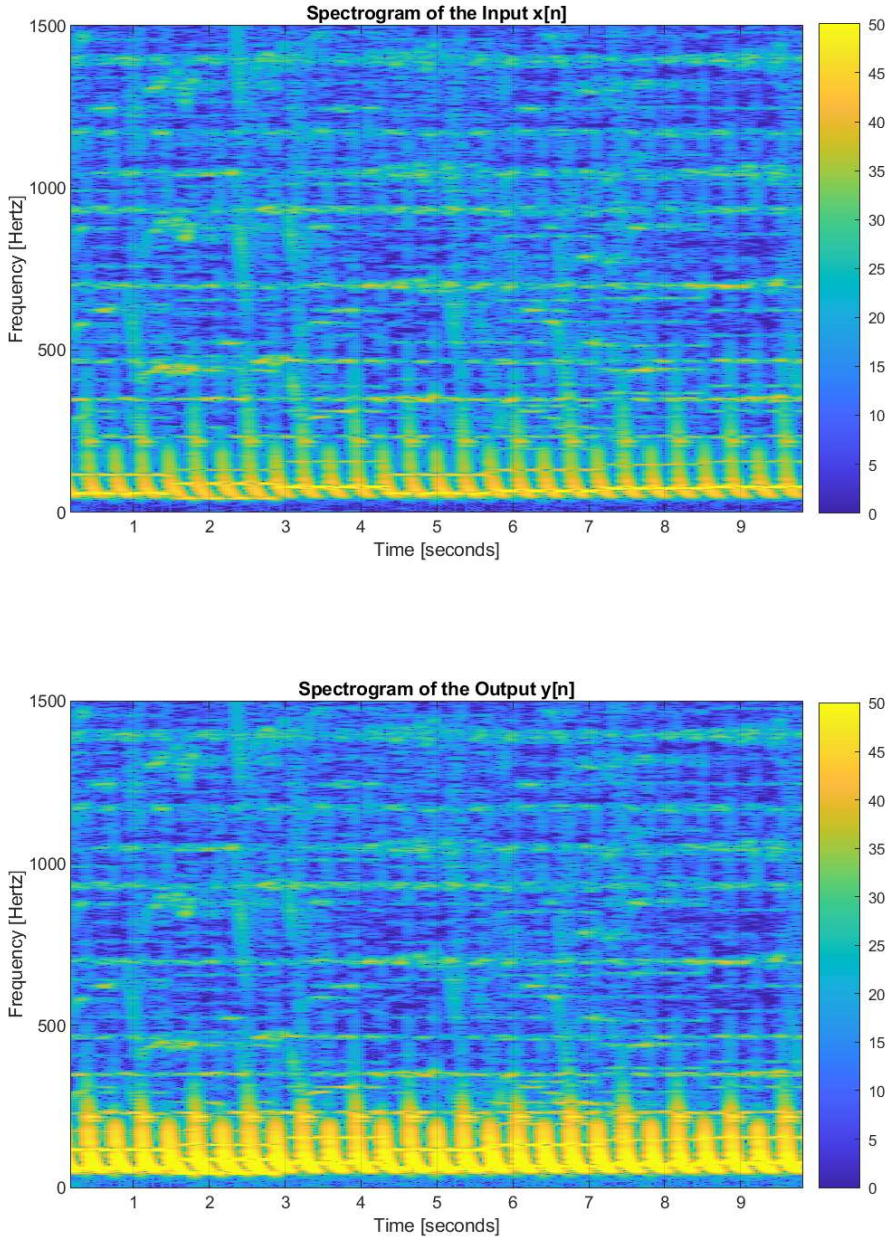


Figure 6: Spectrogram plots of the input audio and the filter output.

2. Using the `freqz` function, plot the filter magnitude response using the sample rate in the audio file.

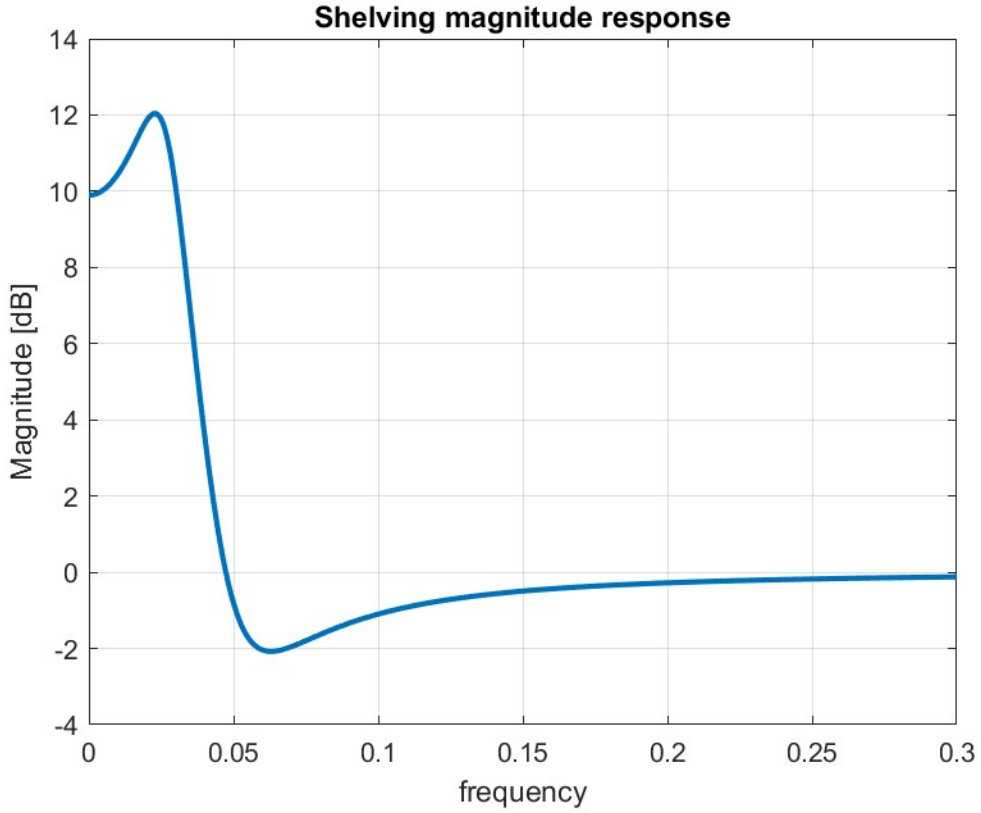


Figure 7: plot of the magnitude response of the shelving filter.

3. What audible effect does the shelving filter impart to the signal? How does the magnitude response of the shelving filter predict this result?

Looking at the magnitude response, the filter "boosts" a specific range of low frequency notes while leaving the other frequencies unattenuated. This causes the bass in the song to be slightly louder than other notes in the higher frequencies.

7. Design a notch filter to cut out the note Bb4=A#4 which has a frequency of 466.16 Hz (https://pages.mtu.edu/~suits/notefreqs.html). This note occurs multiple times in the introduction to the song "Fireflies" by Owl City. Here is an audio clip. To design the notch filter, you will need the audio sample rate and that can be obtained from the audio file using the Matlab command shown below. Process the audio clip using your IIR filtering code and make spectrogram plots of the input signal x[n] and the output signal y[n]. Can you see that the notch filter removed the notes at 466.16 Hz?

Viewing Figure 7 the notes at the frequency 466.16 Hz are getting attenuated in the output of the filter.

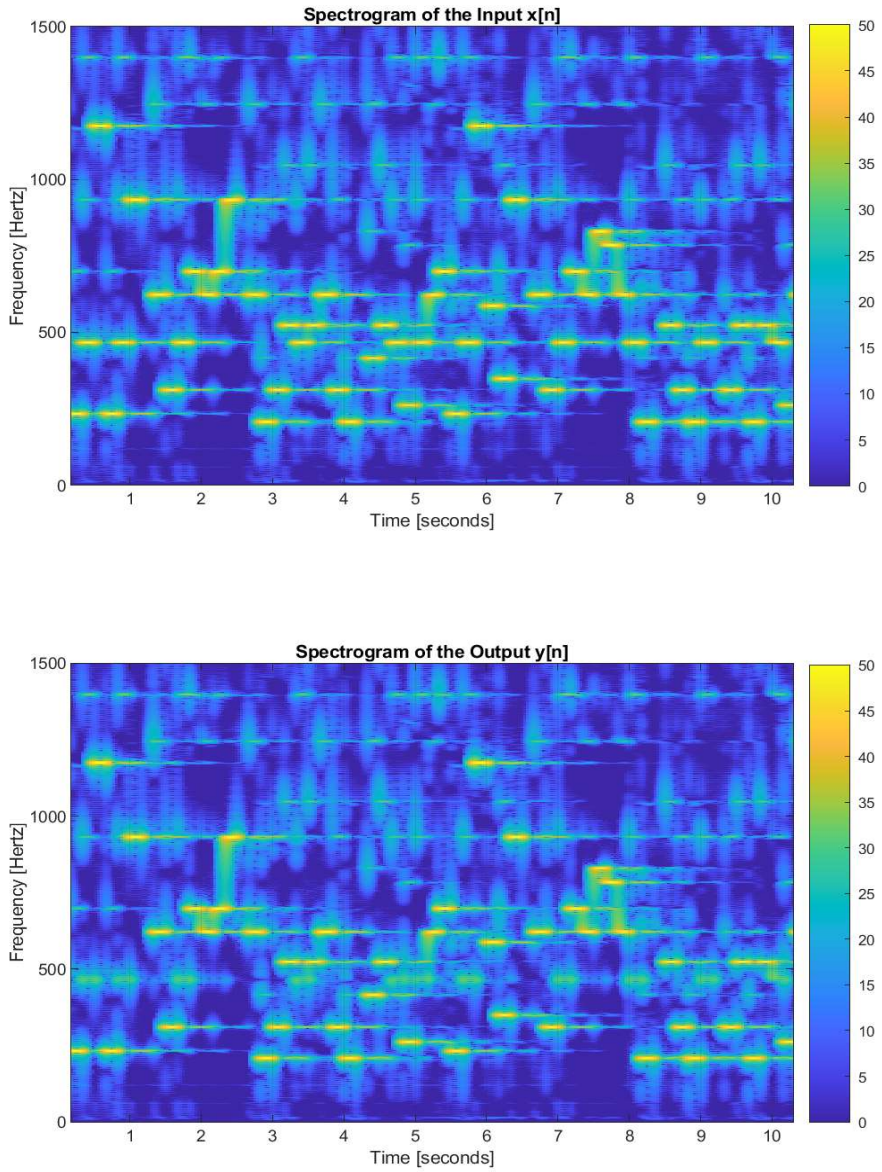


Figure 8: Spectrogram plots of the firefly intro pre filter and post notch filter

```
function y = myFIRfilter(b,x)
persistent z L; % Persistent internal filter memory
if isempty(z)
    z = zeros(size(b)); % Initialize to zero
    L = length(b);
end

    y = 0;
    for n = L:-1:2
        z(n) = z(n-1);
        y = y + b(n) * z(n);
    end
    z(1) = x;
    y = y + z(1)*b(1);
end
```

```
clear
clc

%% problem 2
h = [1 2 3 4 4 3 2 1];
x = [1 0 0 0 0 0 0 0];
y = zeros([1 8]);

for n = 1:8
    y(n) = myFIRfilter(h,x(n));
end

n = 1:8;
subplot(2,1,1)
stem(n,h)
xlabel('n');
ylabel('h[n]');
title('impulse response');
grid on
n = 1:8;
subplot(2,1,2)
stem(n,y)
xlabel('n');
ylabel('y[n]');
title('filter response');
grid on
shg
saveas(gcf,'myFIRfilter_test','png');

%% problem 3
clear myFIRfilter
n = 0:20;
H = [.0625, .2764, .4182, .2764, .0625];
x = cos(2 * pi * 0.422 .* n);
y = zeros([1 21]);

for m = 1:21
    y(m) = myFIRfilter(H, x(m));
end

subplot(2, 1, 1)
plot(n,x);
title('prefilter');
xlabel('n')
ylabel('x[n]');
subplot(2,1,2);
plot(n, y);
title('postfilter')
xlabel('n')
ylabel('y[n]')
saveas(gcf, 'problem 3', 'png');
```

```
%% problem 4

%get impulse responses
lpf_1000 = readmatrix('1000Hz_lowpass.txt');
lpf_2000 = readmatrix('2000Hz_lowpass.txt');
lpf_1000 = lpf_1000';
lpf_2000 = lpf_2000';

% Design all pass filter
L = 50;
N = 101;
hall = zeros(size(lpf_2000));
hall(L+1) = 1; % This is the "delta"
%band stop
bsf = hall - lpf_2000 + lpf_1000;

[BSF, F] = freqz(bsf, 1, 2^14, 1);
plot(F,abs(BSF),'linewidth', 2);
xlabel('frequency');
ylabel('Magnitude [linear]')
title('Magnitude Response of Bandstop')
grid on
shg;
saveas(gcf,'my_Bandstop_filter','png');
```

```
function y = myIIRfilter(b,a,x)
persistent zx zy Lx Ly; % Persistent internal filter memory
if isempty(zx)
    zx = zeros(size(b)); % Initialize input buffer to zero
    zy = zeros(size(a)); % Initialize output buffer zero
    Lx = length(b);
    Ly = length(a);
end

y = 0;
    for n = Lx:-1:2
        zx(n) = zx(n-1);
        y = y + b(n) * zx(n);
    end
zx(1) = x;
y = y + zx(1)*b(1);

    for n = Ly:-1:2
        zy(n) = zy(n-1);
        y = y - a(n)*zy(n);
    end
zy(1) = y;
end
```

```
clear
clc
clear myIIRfilter

%% problem 5
b = [1.0082 -1.9784 0.9727];
a = [1 -1.9793, 0.9801];

[x,Fs] = audioread('rainbow_road_snip_original.wav');
y = zeros([1 length(x)]);

for n = 1:length(x)
    y(n) = myIIRfilter(b,a,x(n));
end


NFFT = 2^14;
figure();
subplot(211);
[S,F,T] = spectrogram(x,hamming(NFFT),round(0.9*NFFT),NFFT,Fs);
imagesc(T,F,20*log10(abs(S)),[0 50]);
colorbar;
set(gca,'YDir','normal');
ylim([0 1500]);
xlabel('Time [seconds]');
ylabel('Frequency [Hertz]');
title('Spectrogram of the Input x[n]');
grid on;

subplot(212);
[S,F,T] = spectrogram(y,hamming(NFFT),round(0.9*NFFT),NFFT,Fs);
imagesc(T,F,20*log10(abs(S)),[0 50]);
colorbar;
set(gca,'YDir','normal');
ylim([0 1500]);
xlabel('Time [seconds]');
ylabel('Frequency [Hertz]');
title('Spectrogram of the Output y[n]');
grid on;
shg;
orient tall;
print -dpng audio_spectrograms.png

figure
[H_shelve, F] = freqz(b,a,Fs);
plot(F,20*log10(abs(H_shelve)),'linewidth', 2);
xlabel('frequency');
ylabel('Magnitude [dB]')
xlim([0 .3]);
title('Shelving magnitude response')
grid on
```

```
shg;
saveas(gcf,'my_shelving_filter','png');

%% Problem 6


[x,Fs] = audioread('fireflyintro.wav');
N = length(x);

q = exp(-20*pi/Fs);
Fn = 466.16;
fn = Fn/Fs;
wn = 2 * pi * fn;

b = [1, -2*cos(wn), 1];
a = [1, -2*q*cos(wn), q^2];

y = zeros([1 N]);
for n = 1:N
    y(n) = myIIRfilter(b, a, x(n));
end

NFFT = 2^14;
figure();
subplot(211);
[S,F,T] = spectrogram(x,hamming(NFFT),round(0.9*NFFT),NFFT,Fs);
imagesc(T,F,20*log10(abs(S)),[0 50]);
colorbar;
set(gca,'YDir','normal');
ylim([0 1500]);
xlabel('Time [seconds]');
ylabel('Frequency [Hertz]');
title('Spectrogram of the Input x[n]');
grid on;

subplot(212);
[S,F,T] = spectrogram(y,hamming(NFFT),round(0.9*NFFT),NFFT,Fs);
imagesc(T,F,20*log10(abs(S)),[0 50]);
colorbar;
set(gca,'YDir','normal');
ylim([0 1500]);
xlabel('Time [seconds]');
ylabel('Frequency [Hertz]');
title('Spectrogram of the Output y[n]');
grid on;
shg;
orient tall;
print -dpng audio_spectrograms2.png
```

```
clear
clc

% Filter Specification
Fs = 8000; % samples/second
% Cut off frequencies
Fpass = 2000; % Hz
Fstop = 2100; % Hz
% Filter order
N = 101; % Length of filter impulse response (odd integer)

% FIR Filter Design Procedure
fpass = Fpass/Fs;
fstop = Fstop/Fs;
f1 = (fstop + fpass)/2;
f2 = (fstop - fpass)/2;
L = (N-1)/2;
n = [-L:L];
h = (sin(2*pi*f1*n) .* sin(2*pi*f2*n)) ./ ((pi*n) .* (2*pi*f2*n));
h(L+1) = 2*f1;

% Plot magnitude response
[H,F] = freqz(h,1,2^14,Fs); % Compute frequency response
subplot(211);
plot(F,abs(H),'LineWidth',2); % Plot magnitude response [linear]
xlabel('Frequency [Hertz]');
ylabel('Magnitude [linear]');
xlim([0 Fs/2]);
grid on;
subplot(212);
plot(F,20*log10(abs(H)),'LineWidth',2); % Plot magnitude response [dB]
xlabel('Frequency [Hertz]');
ylabel('Magnitude [dB]');
ylim([-80 5]); xlim([0 Fs/2]);
grid on; shg;
writematrix(h','2000Hz_lowpass.txt' );
```